

TikTok Manager - التوثيق الكامل

دليل شامل لفهم كل جزء من المشروع

توثيق مشروع TikTok Manager

فهرس المحتويات

1. مقدمة عن المشروع
2. المكدس التكنولوجي (Tech Stack)
3. هيكل المشروع الكامل
4. ملفات الإعدادات (Config Files)
5. المجلد `src/types`
6. المجلد `src/services`
7. المجلد `src/store`
8. المجلد `src/hooks`
9. المجلد `src/config`
10. المجلد `src/components`
11. المجلد `src/pages`
12. الملف `App.tsx`
13. الملف `main.tsx`
14. ملفات CSS
15. كيفية إضافة صفحة جديدة
16. كيفية ربط ال Backend (Laravel)

1. مقدمة عن المشروع

هذا المشروع هو **تطبيق ويب** (Web Application) لإدارة حسابات TikTok. يسمح لك بمشاهدة إحصائيات الحساب، إدارة المنشورات، وتغيير الإعدادات.

ما هو React؟

React هي مكتبة **JavaScript** لبناء واجهات المستخدم (UI). الفكرة الأساسية فيها هي تقسيم الصفحة إلى **مكونات** (Components). كل مكون هو قطعة مستقلة من الواجهة.

مثال بسيط:

```
function Button() {
  return <button>Click me</button>
}
```

هذا مكون (Component) اسمه **Button** يرجع زر HTML. أي مكون يرجع JSX (HTML داخل JavaScript).

ما هو JSX؟

JSX هو هجين بين HTML و JavaScript. يسمح لك بكتابة HTML مباشرة داخل الكود:

```
const name = "Ahmed"
return <h1>Hello {name}</h1> // النتيجة: Hello Ahmed
```

الأقواس **{ }** تستخدم لإدخال متغيرات JavaScript داخل HTML.

ما هو TypeScript؟

TypeScript هو **JavaScript مع أنواع** (Types). يضيف أنواعًا للمتغيرات والدوال لتصيد الأخطاء قبل تشغيل البرنامج.

مقارنة:

```
// JavaScript
function add(a, b) {
  return a + b
}
add("5", 3) // "53" ← خطأ! جمع نص مع رقم
```

```
// TypeScript
function add(a: number, b: number): number {
  return a + b
}
add("5", 3) // ❌ الخطأ يظهر قبل التشغيل
```

ما هو Vite؟

Vite هو أداة بناء (Build Tool). يقوم بمهام:

- **Dev Server**: خادم تطوير يعيد تحميل الصفحة تلقائيًا عند التعديل (HMR)
- **Bundler**: يحول الكود إلى ملفات نهائية للنشر (Production Build)

كيف يعمل تدفق المشروع؟

```
(Browser) المتصفح
↓
index.html ← الصفحة الرئيسية
↓
main.tsx ← نقطة الدخول
↓
App.tsx ← المكون الرئيسي (التوجيه)
↓
Pages ← (Dashboard, Posts, Settings, Login) الصفحات
↓
Components ← (Sidebar, StatsCard) المكونات
↓
Store ← (Zustand) البيانات
↓
Services ← API / Mock
```

2. المكّس التكنولوجي (Tech Stack)

المكتبات المستخدمة:

المكتبة	الدور	لماذا اخترناها؟
React 18	بناء واجهة المستخدم	الأكثر شهرة، مجتمع كبير
TypeScript	أنواع للكود	يمنع الأخطاء، توثيق ذاتي
Vite 5	بناء وتشغيل المشروع	CRA سريع جدًا مقارنة بـ
Tailwind CSS 4	التنسيق (Styling)	لا ملفات HTML، داخل CSS أكواد منفصلة
React Router 6	التنقل بين الصفحات	سهل وحديث routing
Zustand	إدارة الحالة (State Management)	Boilerplate بسيط، خفيف، بدون
Zod	التحقق من صحة البيانات (Validation)	TypeScript أنواع ذكية، يتكامل مع
Recharts	رسم المخططات البيانية	سهل الاستخدام، React مبني على
Axios	HTTP طلبات	interceptors يدعم، fetch أفضل من

شرح مبسط لكل مكتبة:

Tailwind CSS

بدل ما تكتب CSS في ملف منفصل:

```
/* CSS العادي */
.button {
  background-color: red;
  padding: 10px;
  border-radius: 5px;
}
```

تكتب كل شيء كـ **class** في HTML:

```
// Tailwind
<button className="bg-[#fe2c55] p-2 rounded-lg">Click</button>
```

. **bg-[#fe2c55]** = background-color, **p-2** = padding, **rounded-lg** = border-radius

Zustand

مكتبة لحفظ البيانات التي تشارك بين عدة مكونات. مثال:

```
// تعريف المخزن (Store)
const useCounterStore = create((set) => ({
  count: 0,
  increase: () => set((state) => ({ count: state.count + 1 })),
}))

// استخدامه في أي مكون
function MyComponent() {
  const count = useCounterStore((state) => state.count)
  const increase = useCounterStore((state) => state.increase)
  return <button onClick={increase}>Count: {count}</button>
}
```

Zod

مكتبة للتحقق من صحة البيانات (Validation):

```
import { z } from 'zod'

// تعريف "قانون" للبيانات
const UserSchema = z.object({
  email: z.string().email(), // يجب أن يكون إيميل صحيح
  age: z.number().min(18).max(120), // بين 18 و 120
})

// التحقق
const result = UserSchema.safeParse({ email: "test", age: 15 })
if (!result.success) {
  console.log(result.error.issues) // يطلع الأخطاء
}
```

Recharts

مكتبة رسوم بيانية. تعمل بمبدأ المكونات:

```
<LineChart width={500} height={300} data={myData}>
  <XAxis dataKey="date" />
  <YAxis />
  <Line type="monotone" dataKey="views" stroke="#fe2c55" />
</LineChart>
```

كل جزء من الرسم البياني هو مكون `Line` , `XAxis` , `LineChart` , إلخ. React:

Axios

مكتبة لطلبات HTTP (للتواصل مع السيرفر):

```
import axios from 'axios'

// GET request
const { data } = await axios.get('/api/users')

// POST request
const { data } = await axios.post('/api/login', {
  email: "test@test.com",
  password: "123456"
})
```

```

TikTokManager/
|
├─ index.html          # HTML الصفحة الرئيسية
├─ package.json        # معلومات المشروع والمكتبات
├─ vite.config.ts      # إعدادات Vite
├─ tsconfig.json       # الرئيسية TypeScript إعدادات
├─ tsconfig.app.json   # للتطبيق TS إعدادات
├─ tsconfig.node.json  # Node.js لـ TS إعدادات
|
├─ config/             # إعدادات قديمة (قديم)
|   └─ colors.ts
|   └─ languages/
|
├─ public/             # الملفات الثابتة (صور، إلخ)
|
├─ src/               # كل كود المصدر هنا
|   |
|   └─ main.tsx        # نقطة دخول التطبيق
|   └─ App.tsx         # المكون الرئيسي (التوجيه)
|   └─ App.css         # تنسيق إضافي
|   └─ index.css       # Tailwind + تنسيق عام
|   └─ vite-env.d.ts   # Vite تعريفات
|   |
|   └─ types/          # (TypeScript) تعريفات الأنواع
|       └─ index.ts
|   |
|   └─ services/       # طبقة الخدمات (API + Mock)
|       └─ api.ts       # Axios instance (للاستخدام الحقيقي)
|       └─ mock.ts      # بيانات وهمية (للتطوير)
|   |
|   └─ store/          # (Zustand) إدارة الحالة
|       └─ index.ts
|   |
|   └─ hooks/          # (Hooks) دوال مساعدة
|       └─ useTranslation.ts
|   |
|   └─ config/         # إعدادات التطبيق

```



```

| | └─ colors.ts          # ألوان التطبيق
| | └─ languages/        # ملفات الترجمة
| |   └─ en.json
| |   └─ fr.json
| |
| └─ components/         # مكونات قابلة لإعادة الاستخدام
|   └─ Sidebar.tsx       # الشريط الجانبي
|   └─ StatsCard.tsx     # بطاقة إحصائية
|   └─
└─ pages/                # صفحات التطبيق
    └─ Login.tsx
    └─ Dashboard.tsx
    └─ Posts.tsx
    └─ Settings.tsx
└─ DOCUMENTATION_AR.md   # هذا الملف 😊

```

شرح كل مجلد:

src/types 📁

لتعريف أنواع البيانات (Interfaces). مثل شكل المستخدم، المنشور، الإحصائيات.

src/services 📁

طبقة الخدمات. تحتوي على:

- **api.ts**: إعدادات Axios للتواصل مع السيرفر الحقيقي (Laravel)
- **mock.ts**: بيانات وهمية تحاكي السيرفر (للتطوير بدون Backend)

src/store 📁

المخازن (Stores). باستخدام Zustand لحفظ البيانات المشتركة.

src/hooks 📁

الدوال المساعدة. حاليًا فيه **useTranslation** للترجمة بين العربية والإنجليزية والفرنسية.

src/config 📁

الإعدادات الثابتة. ألوان التطبيق وملفات الترجمة.

src/components 📁

المكونات القابلة لإعادة الاستخدام. مثل Sidebar و StatsCard.

[src/pages](#) 📁

صفحات التطبيق. كل صفحة كاملة تمثل Route معين.

4. ملفات الإعدادات (Config Files)

4.1 [index.html](#)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>TikTok Manager</title>
  </head>
  <body>
    <div id="root"></div>           هنا سيتم وضع التطبيق ←
    <script type="module" src="/src/main.tsx"></script> ← الرئيسي JavaScript ملف
  </body>
</html>
```

- `lang="en"` : لغة الصفحة (يمكن تغييرها لـ "ar")
- `root#` : هو elemento فارغ، React ستقوم بملئه بكل المحتوى
- `<script ... src="/src/main.tsx">` : هذا هو ملف JavaScript الذي يحتوي على كود React

ملاحظة مهمة: Vite يتعامل مع `tsx.` مباشرة. لا نحتاج لتحويله لـ JavaScript يدويًا.

```

{
  "name": "tiktok-manager",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",           // تشغيل خادم التطوير
    "build": "vite build",   // بناء النسخة النهائية
    "preview": "vite preview" // معاينة النسخة النهائية
  },
  "dependencies": {          // مكتبات الإنتاج
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.0",
    "zustand": "...",
    "axios": "...",
    "zod": "...",
    "recharts": "..."
  },
  "devDependencies": {       // مكتبات التطوير فقط
    "vite": "^5.4.0",
    "@vitejs/plugin-react": "^4.3.1",
    "tailwindcss": "...",
    "@tailwindcss/vite": "...",
    "typescript": "...",
    "@types/react": "...",
    "@types/react-dom": "..."
  }
}

```

- `"type": "module"`: يعني أننا نستخدم ES Modules (import/export) بدل CommonJS (require)
- `scripts`: أوامر يمكن تشغيلها بـ `npm run dev` أو `npm run build`
- `dependencies`: مكتبات يحتاجها التطبيق في الإنتاج
- `devDependencies`: مكتبات يحتاجها المطور فقط (مثل TypeScript)

vite.config.ts 4.3

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

- إضافة دعم React (تحويل JSX إلى JavaScript): `()react`
- إضافة دعم Tailwind CSS: `()tailwindcss`

tsconfig.json 4.4

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" }, // إعدادات التطبيق
    { "path": "./tsconfig.node.json" } // إعدادات Node.js
  ]
}
```

نستخدم **Project References** في TypeScript. نفصل إعدادات التطبيق عن إعدادات Node.js لأن كل واحد له هدف مختلف.

tsconfig.app.json 4.5

```
{
  "compilerOptions": {
    "target": "ES2020",           // ناتج الترجمة: ES2020
    "lib": ["ES2020", "DOM", "DOM.Iterable"], // مكتبات متاحة
    "module": "ESNext",          // نظام الوحدات
    "moduleResolution": "bundler", // حل الوحدات بطريقة Bundler
    "jsx": "react-jsx",          // تحويل JSX
    "strict": true,              // تفعيل كل الفحوصات
    "resolveJsonModule": true,    // استيراد ملفات JSON
    "skipLibCheck": true         // تخطي فحص المكتبات
  },
  "include": ["src"]             // المجلد المطلوب فحصه
}
```

tsconfig.node.json 4.6

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "noEmit": true,
    "strict": true
  },
  "include": ["vite.config.ts"] // فقط ملف vite.config.ts
}
```

تعريف أنواع البيانات — src/types/index.ts

```
// البيانات في التطبيق (Shape) هذا الملف يحدد شكل

export interface User {
  id: string          // معرف فريد
  email: string        // البريد الإلكتروني
  name: string         // الاسم
  avatar?: string      // صورة المستخدم (اختياري: علامة ?)
  tiktokUsername?: string // (اختياري) اسم مستخدم TikTok
}

export interface Post {
  id: string
  description: string      // نص المنشور
  videoUrl?: string        // رابط الفيديو
  thumbnailUrl?: string    // صورة مصغرة
  views: number            // عدد المشاهدات
  likes: number            // عدد الإعجابات
  comments: number         // عدد التعليقات
  shares: number           // عدد المشاركات
  createdAt: string        // تاريخ الإنشاء (ISO string)
  status: 'published' | 'draft' | 'scheduled' // الحالة: قيم محددة
}

export interface Stats {
  followers: number
  followersChange: number // نسبة التغير
  views: number
  viewsChange: number
  likes: number
  likesChange: number
  comments: number
  commentsChange: number
}

export interface DailyStats {
```

```

date: string
views: number
likes: number
comments: number
shares: number
}

```

شرح المصطلحات:

المصطلح	المعنى
<code>interface</code>	(Object) كائن (Shape) طريقة لتعريف شكل
<code>string</code>	نوع نصي
<code>number</code>	نوع عددي
<code>?</code>	علامة اختيارية: الخاصية ممكن تكون أو لا
<code>'published' 'draft'</code>	القيمة تكون واحدة من هذه الخيارات فقط: Union Type

لماذا نحتاج هذا الملف؟

- TypeScript يستخدمه لفحص الأخطاء
- أي programmer جديد يفهم شكل البيانات بسرعة
- المترجم (Compiler) يوقف الأخطاء قبل تشغيل البرنامج

6.1 src/services/api.ts — اتصال السيرفر الحقيقي

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000/api',
  headers: {
    'Content-Type': 'application/json',
  },
})

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('tiktok_token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

export default api
```

الشرح:

- **axios.create()**: ننشئ نسخة مخصصة من Axios مع إعدادات افتراضية:
 - **baseURL**: الرابط الأساسي للسيرفر. يستخدم متغير البيئة **VITE_API_URL** أو localhost افتراضياً
 - **headers**: نرسل البيانات بصيغة JSON
 - **Interceptor : interceptors.request.use()** — دالة تُستدعى قبل كل طلب:
 - يقرأ **tiktok_token** من localStorage
 - إذا وجد التوكن، يضيفه في رأس Authorization
 - هذا يعني أن كل طلب يرسل التوكن تلقائياً
- لماذا **interceptor** بدل ما نضيف التوكن يدوياً لكل طلب؟ لأنه لو كان عندك 100 طلب API، ما تضطرش تضيف التوكن لكل واحد.


```
import type { User, Post, Stats, DailyStats } from '../types'

// Network محاكاة (دالة مساعدة لعمل تأخير)
const delay = (ms: number) =>
  new Promise<void>(resolve => setTimeout(resolve, ms))
```

ما هي الـ **Promise**؟ Promise هي طريقة للتعامل مع العمليات غير المتزامنة (Asynchronous). مثل طلب من السيرفر يستغرق وقتًا.

مقارنة:

```
// يتوقف حتى يكتمل - (Synchronous) متزامن
const data = readFileSync('file.txt')
console.log(data) // يطبع بعد ما يقرأ الملف

// لا يتوقف - (Asynchronous) غير متزامن
readFile('file.txt', (data) => {
  console.log(data) // يطبع لما الملف يجهز
})
console.log("Hello") // يطبع أولاً!
```

:Async/Await

```
async function getData() {
  const data = await fetch('/api/data') // ينتظر هنا
  console.log(data)
}
```

async : تخبر أن الدالة غير متزامنة **await** : تنتظر حتى تكتمل العملية

بقية الملف **mock.ts** :

```
// بيانات المستخدم الافتراضي
let currentUser: User = {
  id: '1',
  email: 'demo@tiktok.com',
  name: 'Demo User',
  avatar: '',
  tiktokUsername: '@demouser',
}

// قائمة المنشورات
let posts: Post[] = [
  {
    id: '1',
    description: 'Check out this new dance challenge! #dance #viral',
    thumbnailUrl: '',
    views: 45200,
    likes: 8200,
    comments: 1340,
    shares: 560,
    createdAt: '2026-07-10T14:30:00Z',
    status: 'published',
  },
  // المزيد من المنشورات ...
]

// الإحصائيات
let stats: Stats = { ... }

// إحصائيات يومية للرسم البياني
let dailyStats: DailyStats[] = [ ... ]
```

ثم نعرّف **الدوال** التي تحاكي API:

```

export const mockAuth = {
  login: async (email: string, password: string): Promise<User> => {
    await delay(800) // 800 ms (محاكاة Network)

    // تحقق بسيط
    if (email !== 'demo@tiktok.com' || password !== 'demo') {
      throw new Error('Invalid credentials')
    }

    return currentUser
  },

  logout: async (): Promise<void> => {
    await delay(300)
  },
}

export const mockDashboard = {
  getStats: async (): Promise<Stats> => {
    await delay(600)
    return stats
  },
  getDailyStats: async (): Promise<DailyStats[]> => {
    await delay(600)
    return dailyStats
  },
}

export const mockPosts = {
  getAll: async (): Promise<Post[]> => {
    await delay(500)
    return posts
  },

  create: async (data: Partial<Post>): Promise<Post> => {
    await delay(400)
    const newPost: Post = {
      id: String(Date.now()), // ID فريد
      description: data.description || '',
    }
  },
}

```

```

    views: 0,
    likes: 0,
    comments: 0,
    shares: 0,
    createdAt: new Date().toISOString(),
    status: 'draft',
    ...data // ندمج البيانات الجديدة
  }
  posts = [newPost, ...posts] // نضيف في البداية
  return newPost
},

remove: async (id: string): Promise<void> => {
  await delay(300)
  posts = posts.filter(p => p.id !== id) // نحذف المنشور
},
}

```

شرح `...data` :Spread Operator

```

const obj1 = { name: 'Ahmed', age: 25 }
const obj2 = { ...obj1, age: 30 }
// النتيجة : { name: 'Ahmed', age: 30 } - age تم تعديل obj1 تم دمج

```

شرح `filter` :

```

const numbers = [1, 2, 3, 4, 5]
const bigNumbers = numbers.filter(n => n > 3)
// النتيجة : [5, 4]

```

لماذا `mock.ts` ؟

- نطور بدون Backend
- نختبر كل الوظائف
- لما يجي الـ Backend الحقيقي، نغير فقط ملف `mock.ts` → نستخدم `api.ts`
- الصفحات (Pages) لا تحتاج أي تغيير لأنها تستخدم نفس الدوال

— إدارة الحالة (State Management) src/store/index.ts

ما هي "الحالة" (State)؟ الحالة هي البيانات التي يتغير محتواها أثناء استخدام التطبيق. مثل:

- هل المستخدم مسجل دخول؟ (true/false)
- ما هي المنشورات؟ (قائمة تتغير)
- ما هي اللغة المختارة؟ ('fr' أو 'en')

لماذا نحتاج **State Management**؟ لأن المكونات المختلفة تحتاج لنفس البيانات. مثال: **Sidebar** يحتاج لمعرفة إذا المستخدم مسجل دخول، و **App** يحتاج نفس المعلومة. بدل ما نمرر البيانات من أب لابن (Prop Drilling)، نستخدم مخزن مركزي.

كيف يعمل Zustand؟

```
import { create } from 'zustand'

// create ينشئ مخزن (Store)
const useMyStore = create((set) => ({
  count: 0, // حالة
  increment: () => set((state) => ({ count: state.count + 1 })), // دالة لتغيير الحالة
}))

// الاستخدام
function Component() {
  const count = useMyStore((s) => s.count) // نقرأ الحالة
  const increment = useMyStore((s) => s.increment) // نأخذ الدالة
  return <button onClick={increment}>{count}</button>
}
```

كود المخزن الحقيقي:

```

import { create } from 'zustand'
import type { User, Post, Stats, DailyStats } from '../types'
import { mockAuth, mockDashboard, mockPosts } from '../services/mock'

// --- Auth Store (المصادقة) ---
export interface AuthStore {
  user: User | null // إذا لم يسجل null أو المستخدم الحالي
  isAuthenticated: boolean // هل سجل دخول؟
  isLoading: boolean // هل العملية جارية؟
  login: (email: string, password: string) => Promise<void>
  logout: () => Promise<void>
}

export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  isAuthenticated: false,
  isLoading: false,

  login: async (email, password) => {
    set({ isLoading: true }) // نبدأ التحميل
    try {
      const user = await mockAuth.login(email, password) // mock نطلب من
      set({ user, isAuthenticated: true, isLoading: false }) // نحفظ
    } catch {
      set({ isLoading: false }) // خطأ
      throw new Error('Invalid credentials')
    }
  },

  logout: async () => {
    await mockAuth.logout()
    set({ user: null, isAuthenticated: false }) // نمسح البيانات
  },
}))

```

شرح **set**:

- **set** هي دالة تغير الحالة
- **set({ isLoading: true })** → يغير الخاصية فقط، الباقي يبقى كما هو

```
// --- Dashboard Store ---
export const useDashboardStore = create<DashboardStore>((set) => ({
  stats: null,
  dailyStats: [],
  isLoading: false,

  fetchStats: async () => {
    set({ isLoading: true })
    const [stats, dailyStats] = await Promise.all([ // طلبين متوازيين
      mockDashboard.getStats(),
      mockDashboard.getDailyStats(),
    ])
    set({ stats, dailyStats, isLoading: false })
  },
}))
```

شرح Promise.all: نرسل طلبين في نفس الوقت بدل الواحد تلو الآخر:

```
// بطيء: كل طلب ينتظر الثاني
const stats = await getStats()
const daily = await getDailyStats() // يبدأ بعد ما يكمل الأول

// سريع: الطلبين مع بعض
const [stats, daily] = await Promise.all([getStats(), getDailyStats()])
```

```
// --- Posts Store ---
export const usePostsStore = create<PostsStore>((set, get) => ({
  posts: [],
  isLoading: false,

  fetchPosts: async () => {
    set({ isLoading: true })
    const posts = await mockPosts.getAll()
    set({ posts, isLoading: false })
  },

  addPost: async (data) => {
    const post = await mockPosts.create(data)
    set({ posts: [post, ...get().posts] }) // نضيف الجديد للقائمة
  },

  deletePost: async (id) => {
    await mockPosts.remove(id)
    set({ posts: get().posts.filter(p => p.id !== id) })
  },
}))
```

شرح **get** :

- **get()** تجلب الحالة الحالية
- نحتاجها في **addPost** و **deletePost** لأننا نريد تعديل القائمة الموجودة

```
// --- Settings Store ---
export const useSettingsStore = create<SettingsStore>((set) => ({
  language: (localStorage.getItem('lang') as 'en' | 'fr') || 'en',

  setLanguage: (lang) => {
    localStorage.setItem('lang', lang) // نحفظ في المتصفح
    set({ language: lang })           // نحدث الحالة
  },
}))
```

شرح **localStorage** :

- localStorage يخزن بيانات في المتصفح تبقى حتى لو أغلقت الصفحة
- `getItem` يقرأ، `setItem` يكتب
- نستخدمه لحفظ اللغة المختارة

لماذا 4 مخازن منفصلة وليس مخزن واحد؟

- أفضل للأداء (كل مكون يتتبع فقط المخزن الذي يحتاجه)
- أسهل للصيانة
- كل مخزن مسؤول عن جزء معين

8. المجلد `src/hooks`

نظام الترجمة — `src/hooks/useTranslation.ts`

```
import en from '../config/languages/en.json' // نستورد الترجمة الإنجليزية
import fr from '../config/languages/fr.json' // نستورد الترجمة الفرنسية
import { useSettingsStore } from '../store' // نستورد مخزن الإعدادات

// نقوم بإنشاء كائن يربط اسم اللغة بملفها
const translations: Record<string, typeof en> = { en, fr }

export function useTranslation() {
  // نقرأ اللغة الحالية من المخزن
  const lang = useSettingsStore((s) => s.language)

  // نرجع ملف الترجمة المناسب
  return translations[lang]
}
```

الاستخدام في المكونات:

```
function MyComponent() {
  const t = useTranslation() // هذا كائن يحتوي كل النصوص

  return (
    <div>
      <h1>{t.app.title}</h1>           // "TikTok Manager"
      <p>{t.login.title}</p>           // "Sign in" أو "Connexion"
    </div>
  )
}
```

لماذا **useTranslation** هو **Hook**؟ لأنه يستخدم Hook آخر (**useSettingsStore**). في React، أي دالة تستخدم **Hooks** يجب أن تكون **Hook** (تبدأ بـ **use**).

ما هو ملف **JSON** للترجمة؟

```
{
  "app": {
    "title": "TikTok Manager"
  },
  "sidebar": {
    "dashboard": "Dashboard",
    "posts": "Posts",
    "settings": "Settings"
  }
}
```

JSON هو تنسيق نصي بسيط لتخزين البيانات. نستخدمه لأن:

- سهل القراءة والكتابة
- تدعمه كل لغات البرمجة
- خفيف وسريع

```
export const colors = {
  primary: '#fe2c55',      // الأساسي TikTok أحمر
  secondary: '#25f4ee',    // سماوي (ثانوي)
  dark: '#121212',        // خلفية داكنة
  darkCard: '#1e1e1e',    // خلفية البطاقات
  darkBorder: '#2e2e2e',  // لون الحدود
  textPrimary: '#ffffff',  // نص أبيض
  textSecondary: '#888888', // نص رمادي
  success: '#00c853',     // أخضر (نجاح)
  warning: '#ffc107',     // أصفر (تحذير)
  error: '#ff1744',       // أحمر (خطأ)
}
```

نستخدم هذه الألوان في Tailwind بهذه الطريقة:

```
<div className="bg-[#121212] text-[#fe2c55]">
```

هذه الملفات تحتوي نصوص التطبيق بلغتين. الهيكل هو نفسه، الاختلاف في القيم.

```

import { NavLink } from 'react-router-dom'
import { useAuthStore, useSettingsStore } from '../store'
import { useTranslation } from '../hooks/useTranslation'

// تعريف روابط التنقل
const navItems = [
  { to: '/', labelKey: 'dashboard' },
  { to: '/posts', labelKey: 'posts' },
  { to: '/settings', labelKey: 'settings' },
] as const

function Sidebar() {
  // نقرأ الحالة من المخازن
  const { isAuthenticated, logout } = useAuthStore()
  const t = useTranslation()
  const lang = useSettingsStore((s) => s.language)

  return (
    <aside className="w-64 h-screen bg-[#121212] border-r border-[#2e2e2e] flex flex-col">

      { /* اللوجو */ }
      <div className="p-6">
        <h1 className="text-xl font-bold text-[#fe2c55]">{t.app.title}</h1>
      </div>

      { /* روابط التنقل */ }
      <nav className="flex-1 px-3 space-y-1">
        {navItems.map((item) => (
          <NavLink
            key={item.to}
            to={item.to}
            end={item.to === '/'}
            className={({ isActive }) =>
              `flex items-center px-4 py-2.5 rounded-lg text-sm font-medium
              transition-colors ${

```

```

      isActive
      ? 'bg-[#fe2c55] text-white' // الرابط النشط
      : 'text-[#888888] hover:text-white hover:bg-[#1e1e1e]' // العادي
    }`
  }
}
>
{t.sidebar[item.labelKey as keyof typeof t.sidebar]}
</NavLink>
)}}
</nav>

{/* قسم الخروج/الدخول */}
<div className="p-3 border-t border-[#2e2e2e]">
  {isAuthenticated ? (
    <button onClick={logout}>
      {t.sidebar.logout}
    </button>
  ) : (
    <NavLink to="/login">
      {t.sidebar.login}
    </NavLink>
  )}
</div>
</aside>
)
}

```

شرح NavLink :

- NavLink مثل Link لكنه يعطيك `isActive` لتنسيق الرابط النشط
- الرابط : `"/"=to`
- `end` : للرابط الرئيسي / فقط (بدون `settings/`)

شرح `className` ك `Function`:

```
className={({ isActive }) => isActive ? 'active' : 'normal'}
```

هذه **Callback** تعطى كقيمة لـ React Router . `className` يناديها ويعطيها `isActive` .

```
const items = [1, 2, 3]
items.map(n => n * 2) // [2, 4, 6]
```

في React، نستخدم `map` لتحويل مصفوفة إلى مصفوفة من JSX:

```
{users.map(user => <div>{user.name}</div>)}
```

10.2 StatsCard.tsx — بطاقة إحصائية

```
interface StatsCardProps {
  title: string // عنوان البطاقة (مثلاً "Followers")
  value: string // القيمة (مثلاً "12.5K")
  change?: number // نسبة التغير (اختياري)
}

function StatsCard({ title, value, change }: StatsCardProps) {
  return (
    <div className="bg-[#1e1e1e] border border-[#2e2e2e] rounded-xl p-5">
      <p className="text-sm text-[#888888] mb-1">{title}</p>
      <p className="text-2xl font-bold text-white">{value}</p>

      {change !== null && ( // موجود change إذا كان
        <span className={change >= 0 ? 'text-green-500' : 'text-red-500'}>
          {change >= 0 ? '+' : ''}{change}%
        </span>
      )}
    </div>
  )
}
```

شرح Props:

- Props هي خصائص تمرر للمكون
- `StatsCardProps` هو TypeScript Interface يحدد شكل ال Props

شرح `(...) && change !== null`• هذه **Conditional Rendering**

- إذا كان `change` موجودًا، نعرض العنصر
- إذا كان `null` أو `undefined`، لا نعرض شيئًا

شرح `condition ? value1 : value2`• **Ternary Operator**

- إذا `condition` صحيح → يرجع `value1`
- إذا خطأ → يرجع `value2`

| **11. المجلد** `src/pages`**11.1 Login.tsx** — صفحة تسجيل الدخول

```
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { z } from 'zod'
import { useAuthStore } from '../store'
import { useTranslation } from '../hooks/useTranslation'

// تعريف قواعد التحقق من صحة البيانات
const loginSchema = z.object({
  email: z.string().email('Invalid email'),
  password: z.string().min(1, 'Password is required'),
})
```

شرح `useState`:

```
const [email, setEmail] = useState('') // email = '', setEmail يغيره
```

- `useState` هو **Hook** يحفظ قيمة تتغير
- يرجع مصفوفة: [القيمة, الدالة التي تغيروها]
- لما تتغير القيمة، React يعيد رسم المكون

```
const [count, setCount] = useState(0)

<button onClick={() => setCount(count + 1)}>
  Clicked {count} times
</button>
```

```
function Login() {
  const t = useTranslation()
  const navigate = useNavigate()
  const { login, isLoading } = useAuthStore()

  // محلية (State) حالة
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault() // نمنع إرسال النموذج الافتراضي
    setError('') // نمنح الأخطاء القديمة

    // التحقق من صحة البيانات
    const result = loginSchema.safeParse({ email, password })

    if (!result.success) {
      setError(result.error.issues[0].message) // أول خطأ
      return
    }

    try {
      await login(email, password)
      navigate('/') // نقل للمصفحة الرئيسية
    } catch {
      setError(t.login.error)
    }
  }
}
```


شرح `e.preventDefault()`:

- لما تضغط Submit في form، المتصفح يعيد تحميل الصفحة
- `e.preventDefault()` يمنع هذا السلوك الافتراضي

شرح `try/catch`:

```
try {
  // كود ممكن يسبب خطأ
  const data = await riskyFunction()
  console.log(data)
} catch (error) {
  // هنا نتعامل مع الخطأ
  console.log('Something went wrong:', error)
}
```

الجزء البصري (JSX):

```

return (
  <div className="min-h-screen bg-[#121212] flex items-center justify-center">
    <div className="bg-[#1e1e1e] border border-[#2e2e2e] rounded-xl p-8 w-full max-w-sm">
      <h1 className="text-2xl font-bold text-white mb-6 text-center">
        {t.login.title}
      </h1>

      <form onSubmit={handleSubmit} className="space-y-4">
        {/* حقل الإيميل */}
        <div>
          <label className="block text-sm text-[#888888] mb-1">
            {t.login.email}
          </label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full px-3 py-2 bg-[#121212] border border-[#2e2e2e]
              rounded-lg text-white text-sm
              focus:outline-none focus:border-[#fe2c55]"
            placeholder="demo@tiktok.com"
          />
        </div>

        {/* حقل كلمة المرور */}
        <div>
          <label className="block text-sm text-[#888888] mb-1">
            {t.login.password}
          </label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="w-full px-3 py-2 bg-[#121212] border border-[#2e2e2e]
              rounded-lg text-white text-sm
              focus:outline-none focus:border-[#fe2c55]"
            placeholder="demo"
          />
        </div>
      </form>
    </div>
  </div>
)

```

```

    { /* رسالة الخطأ */ }

    {error && <p className="text-sm text-[#ff1744]">{error}</p>}

    { /* زر الإرسال */ }
    <button
      type="submit"
      disabled={isLoading}
      className="w-full py-2.5 bg-[#fe2c55] text-white rounded-lg
        text-sm font-medium hover:bg-[#e01e45]
        transition-colors disabled:opacity-50"
    >
      {isLoading ? '...' : t.login.submit}
    </button>
  </form>

  <p className="mt-4 text-xs text-[#555] text-center">
    Demo: demo@tiktok.com / demo
  </p>
</div>
</div>
)

```

شرح **onChange** :

```
<input value={email} onChange={(e) => setEmail(e.target.value)} />
```

- **onChange** يستدعى كل مرة يتغير فيها محتوى input

- **e.target.value** هو النص الجديد في input

- **setEmail** تحدث قيمة **email**

شرح **disabled={isLoading}** : لما **isLoading = true** ، الزر يصبح غير قابل للضغط. نستخدمه لمنع الضغط المتكرر.

```
import { useEffect } from 'react'
import {
  LineChart, Line, XAxis, YAxis,
  CartesianGrid, Tooltip, ResponsiveContainer,
} from 'recharts'
import StatsCard from '../components/StatsCard'
import { useDashboardStore } from '../store'
import { useTranslation } from '../hooks/useTranslation'

function Dashboard() {
  const t = useTranslation()
  const { stats, dailyStats, isLoading, fetchStats } = useDashboardStore()

  // نجلب البيانات عند تحميل الصفحة
  useEffect(() => {
    fetchStats()
  }, [fetchStats])
}
```

شرح **useEffect**:

```
useEffect(() => {
  // هذا الكود يشغل مرة واحدة عند تحميل المكون
  fetchData()
}, []) // المصفوفة الفارغة تعني: مرة واحدة فقط
```

useEffect يشغل كود جانبي (Side Effect) مثل:

- جلب بيانات من API
- تعديل عنوان الصفحة
- الاتصال بقاعدة البيانات

المصفوفة (Dependency Array):

- **[]**: يشغل مرة واحدة عند تحميل المكون
- **[count]**: يشغل كل مرة يتغير **count**
- بدون مصفوفة: يشغل في كل Rendering

```
if (isLoading || !stats) {  
  return <div className="text-center text-[#888888] py-20">Loading...</div>  
}
```

هذا **Loading State** — نعرض رسالة "...Loading" إلى أن تجهز البيانات.

```

return (
  <div>
    <h1 className="text-2xl font-bold text-white mb-6">{t.dashboard.title}</h1>

    {/* شبكة البطاقات الإحصائية */}
    <div className="grid grid-cols-4 gap-4 mb-8">
      <StatsCard
        title={t.dashboard.followers}
        value={stats.followers.toLocaleString()} // 12500 → "12,500"
        change={stats.followersChange}
      />
      <StatsCard title={t.dashboard.views} value={stats.views.toLocaleString()}
        change={stats.viewsChange} />
      <StatsCard title={t.dashboard.likes} value={stats.likes.toLocaleString()}
        change={stats.likesChange} />
      <StatsCard title={t.dashboard.comments} value={stats.comments.toLocaleString()}
        change={stats.commentsChange} />
    </div>

    {/* الرسم البياني */}
    <div className="bg-[#1e1e1e] border border-[#2e2e2e] rounded-xl p-6">
      <h2 className="text-lg font-semibold text-white mb-4">
        {t.dashboard.dailyViews}
      </h2>

      <ResponsiveContainer width="100%" height={300}>
        <LineChart data={dailyStats}>
          <CartesianGrid strokeDasharray="3 3" stroke="#2e2e2e" />
          <XAxis dataKey="date" stroke="#888888" fontSize={12} />
          <YAxis stroke="#888888" fontSize={12} />
          <Tooltip
            contentStyle={{
              backgroundColor: '#1e1e1e',
              border: '1px solid #2e2e2e',
              borderRadius: '8px',
              color: '#fff',
            }}
          />
          <Line

```

```
        type="monotone"
        dataKey="views"
        stroke="#fe2c55"
        strokeWidth={2}
        dot={{ fill: '#fe2c55' }}
      />
    </LineChart>
  </ResponsiveContainer>
</div>
</div>
)
```

شرح مكونات Recharts:

الدور	المكون
(Responsive) يجعل الرسم متجاوب	ResponsiveContainer
الرسم البياني نفسه	LineChart
المحور الأفقي (التواريخ)	XAxis
المحور العمودي (الأرقام)	YAxis
شبكة الخلفية	CartesianGrid
نافذة منبثقة عند تمرير الماوس	Tooltip
خط البيانات	Line

```
import { useEffect, useState } from 'react'
import { usePostsStore } from '../store'
import { useTranslation } from '../hooks/useTranslation'

function Posts() {
  const t = useTranslation()
  const { posts, isLoading, fetchPosts, addPost, deletePost } = usePostsStore()

  // State محلي للفرم
  const [description, setDescription] = useState('')
  const [showForm, setShowForm] = useState(false)

  useEffect(() => {
    fetchPosts()
  }, [fetchPosts])

  const handleAdd = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!description.trim()) return // نصوص فارغة
    await addPost({ description: description.trim() })
    setDescription('')
    setShowForm(false)
  }
}
```

شرح `()trim`:

`" hello ".trim()` // "hello" - يسمح المسافات الزائدة

→ إذا النص فارغ بعد مسح المسافات، نرجع من غير ما نضيف. `()description.trim!`


```

return (
  <div>
    { /* عنوان الصفحة + زر الإضافة */ }
    <div className="flex items-center justify-between mb-6">
      <h1 className="text-2xl font-bold text-white">{t.posts.title}</h1>
      <button onClick={() => setShowForm(!showForm)}>
        {t.posts.add}
      </button>
    </div>

    { /* فورم الإضافة (يظهر/يختفي) */ }
    {showForm && (
      <form onSubmit={handleAdd} className="...">
        <input
          type="text"
          value={description}
          onChange={(e) => setDescription(e.target.value)}
          placeholder={t.posts.description}
        />
        <button type="submit">{t.posts.add}</button>
      </form>
    )}

    { /* قائمة المنشورات */ }
    {posts.length === 0 ? (
      <p className="text-[#888888] text-center py-10">{t.posts.noPosts}</p>
    ) : (
      <div className="space-y-3">
        {posts.map((post) => (
          <div key={post.id} className="bg-[#1e1e1e] border border-[#2e2e2e]
            rounded-xl p-5 flex items-center justify-between">
            <div className="flex-1 min-w-0">
              <p className="text-white text-sm font-medium truncate">
                {post.description}
              </p>
              <div className="flex gap-4 mt-2 text-xs text-[#888888]">
                <span>{t.posts.views}: {post.views.toLocaleString()}</span>
                <span>{t.posts.likes}: {post.likes.toLocaleString()}</span>
                <span>{t.posts.comments}: {post.comments.toLocaleString()}</span>
              </div>
            </div>
          </div>
        )}
      </div>
    )}
  </div>
)

```

```

        <span>{t.posts.status}: {post.status}</span>
      </div>
    </div>
    <button onClick={() => deletePost(post.id)}>
      {t.posts.delete}
    </button>
  </div>
  )})
</div>
})
</div>
)

```

شرح : `key={post.id}`

- React يحتاج مفتاح فريد (key) لكل عنصر في القائمة
- المفتاح يساعد React في تتبع العناصر عند التعديل أو الحذف

11.4 Settings.tsx — صفحة الإعدادات

```

import { useState } from 'react'
import { useAuthStore, useSettingsStore } from '../store'
import { useTranslation } from '../hooks/useTranslation'

function Settings() {
  const t = useTranslation()
  const { user } = useAuthStore()
  const { language, setLanguage } = useSettingsStore()

  // State محلي للنموذج
  const [name, setName] = useState(user?.name || '')
  const [email, setEmail] = useState(user?.email || '')

  const handleSave = (e: React.FormEvent) => {
    e.preventDefault()
    // هنا سنضيف لاحقًا حفظ التغييرات
  }
}

```

شرح `user?.name` :

- **Optional Chaining** — إذا `user` موجود، جلب `name`
 - إذا `user = null` أو `undefined`، يرجع `undefined`
 - نستخدم `'' ||` لجعل القيمة الافتراضية نصًا فارغًا
-

```

import { Routes, Route, Navigate } from 'react-router-dom'
import { useAuthStore } from './store'
import Sidebar from './components/Sidebar'
import Dashboard from './pages/Dashboard'
import Posts from './pages/Posts'
import Settings from './pages/Settings'
import Login from './pages/Login'
import './App.css'

function App() {
  const { isAuthenticated } = useAuthStore()

  // Login إذا لم يسجل دخول: نعرض فقط صفحة
  if (!isAuthenticated) {
    return (
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="*" element={<Navigate to="/login" replace />} />
      </Routes>
    )
  }

  // إذا سجل دخول: نعرض التطبيق الكامل
  return (
    <div className="flex h-screen bg-[#121212]">
      <Sidebar />
      <main className="flex-1 overflow-y-auto p-8">
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/posts" element={<Posts />} />
          <Route path="/settings" element={<Settings />} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </main>
    </div>
  )
}

```

```
)
}
```

شرح React Router:

المستخدم يكتب في المتصفح: `http://localhost:5173/posts`



- **BrowserRouter**: يوفر إمكانية Routing للتطبيق
- **Routes**: يُقَيِّم URL ويختار Route المناسب
- **Route**: يربط URL مع Component
- **Navigate**: يعيد توجيه المستخدم (Redirect)

شرح "*"=path :

- * يعني أي مسار آخر (Catch-all)
- إذا المستخدم كتب رابط غير موجود، نوجهه لـ `login/` أو `/`

لماذا **replace** في **Navigate**؟

- **replace** يستبدل الصفحة الحالية في التاريخ (مايقدرش يرجع بـ Back)

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App'
import './index.css'

// HTML في #root نجد العنصر
// ونرسم التطبيق داخله
ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

شرح `!document.getElementById('root')` :

- **Non-null assertion** — نقول لـ TypeScript: "أنا متأكد أن هذا العنصر موجود"

شرح `React.StrictMode` :

- وضع التطوير الصارم
- في وضع التطوير، يشغل الـ Effects مرتين ليكشف الأخطاء

ترتيب التغليف (Wrapping):

```
React.StrictMode
├── BrowserRouter
│   └── App
│       ├── Sidebar
│       └── Routes
│           ├── Dashboard
│           ├── Posts
│           ├── Settings
│           └── Login
```

14. ملفات CSS

index.css

```
@import "tailwindcss"; /* نستورد Tailwind CSS */

body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, ...;
}

/* تنسيق شريط التمرير (Scrollbar) */
::-webkit-scrollbar {
  width: 6px;
}
::-webkit-scrollbar-track {
  background: #121212;
}
::-webkit-scrollbar-thumb {
  background: #2e2e2e;
  border-radius: 3px;
}
```

App.css

فارغ حاليًا، تارك للتنسيقات المستقبلية.

15. كيفية إضافة صفحة جديدة

لنفترض أننا نريد إضافة صفحة "Analytics":

الخطوة 1: أنشئ الملف `src/pages/Analytics.tsx`

```
function Analytics() {
  return (
    <div>
      <h1 className="text-2xl font-bold text-white mb-6">Analytics</h1>
      <p className="text-[#888888]">Coming soon...</p>
    </div>
  )
}
export default Analytics
```

الخطوة 2: أضف الترجمة في ملفات JSON

في `en.json`:

```
{
  "sidebar": {
    "analytics": "Analytics"
  }
}
```

في `fr.json`:

```
{
  "sidebar": {
    "analytics": "Analytique"
  }
}
```


الخطوة 3: أضف الرابط في `Sidebar.tsx`

```
const navItems = [
  { to: '/', labelKey: 'dashboard' },
  { to: '/analytics', labelKey: 'analytics' }, // ← جديد
  { to: '/posts', labelKey: 'posts' },
  { to: '/settings', labelKey: 'settings' },
]
```

الخطوة 4: أضف ال Route في `App.tsx`

```
import Analytics from './pages/Analytics'

// داخل Routes
<Route path="/analytics" element={<Analytics />} />
```

الخطوة 5 (اختياري): أضف Store جديد

إذا الصفحة تحتاج بيانات من API، أنشئ مخزن جديد في `store/index.ts`.

16. كيفية ربط ال Backend (Laravel)

المرحلة 1: التطوير (حاليًا)

- `services/mock.ts`: يحتوي كل البيانات الوهمية
- `services/api.ts`: موجود لكن غير مستخدم

المرحلة 2: التبديل إلى API الحقيقي

تعديل `services/mock.ts`:

```
// بدل ما نرجع بيانات وهمية، نستخدم api.ts
import api from './api'

export const mockDashboard = {
  getStats: async (): Promise<Stats> => {
    const { data } = await api.get('/dashboard/stats') // طلب حقيقي
    return data
  },
  getDailyStats: async (): Promise<DailyStats[]> => {
    const { data } = await api.get('/dashboard/daily-stats')
    return data
  },
}
```

أو نغير import مباشرة في `store/index.ts`

```
// قبل:
import { mockAuth, mockDashboard, mockPosts } from '../services/mock'

// بعد:
import { apiAuth, apiDashboard, apiPosts } from '../services/api'
```

إعدادات Laravel:

1. أنشئ API routes في (`routes/api.php`) Laravel
2. استخدم Sanctum للمصادقة
3. أضف `VITE_API_URL` في ملف `env.` للتطبيق:

```
VITE_API_URL=http://localhost:8000/api
```

```
// routes/api.php
Route::post('/login', [AuthController::class, 'login']);
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/dashboard/stats', [DashboardController::class, 'stats']);
    Route::get('/posts', [PostController::class, 'index']);
    Route::post('/posts', [PostController::class, 'store']);
    Route::delete('/posts/{id}', [PostController::class, 'destroy']);
});
```

🎯 خلاصة

تدفق البيانات:

```
المستخدم يضغط على زر
↓
onClick → set() → تحديث State
↓
React يعيد رسم المكون (Re-render)
↓
المستخدم يرى النتيجة
```

دورة حياة المكون (Component Lifecycle):

1. Mounting: المكون يظهر لأول مرة ← `useEffect(fn, [])`
2. Updating: البيانات تتغير ← `useEffect(fn, [data])`
3. Unmount: المكون يختفي ← `return () => cleanup`

المصطلحات المهمة:

المصطلح	الترجمة	الشرح
Component	مكون	قطعة مستقلة من الواجهة
Props	خصائص	بيانات تدخل للمكون من الأب
State	حالة	بيانات داخلية للمكون
Hook	خطاف	React (useState, useEffect) دالة تتيح ميزة
Store	مخزن	بيانات مشتركة بين المكونات
Route	مسار	يقابله صفحة URL رابط
API	واجهة برمجية	وسيلة التواصل مع السيرفر
Promise	وعد	عملية غير متزامنة
Async/Await	غير متزامن	طريقة للتعامل مع العمليات البطيئة
Type	نوع	(string, number, ...) يحدد شكل المتغير
Interface	واجهة	(Object Shape) يحدد شكل الكائن

نصيحة أخيرة: ابدأ بفهم mock.ts ← store/index.ts ← Dashboard.tsx ← App.tsx . هذا المسار يغطي 80% من منطق التطبيق. بعدها انتقل للمكونات الأخرى.

للبدء: npm run dev وافتح http://localhost:5173 حساب الدخول: demo / demo@tiktok.com

تم إنشاء هذا الملف بواسطة Opencode تاريخ 12/07/2026